

```

m = int(input("Enter a positive integer for m: "))
n = int(input("Enter a positive integer for n: "))

if m <= 0 or n <= 0:
    print("Both m and n must be positive integers.")
else:
    print(f"You entered m = {m} and n = {n}")

Enter a positive integer for m: 3
Enter a positive integer for n: 2
You entered m = 3 and n = 2

import numpy as np

U = np.zeros((m, n))
print(f"Enter {m*n} elements for the {m}x{n} matrix U:")
for i in range(m):
    for j in range(n):
        U[i, j] = float(input(f"Enter element U[{i+1},{j+1}]: "))

display(U)

Enter 6 elements for the 3x2 matrix U:
Enter element U[1,1]: 2
Enter element U[1,2]: 7
Enter element U[2,1]: 5
Enter element U[2,2]: 3
Enter element U[3,1]: 9
Enter element U[3,2]: 1

array([[2., 7.],
       [5., 3.],
       [9., 1.]])

print(f"Shape of U: {U.shape}")
print(f"Number of dimensions of U: {U.ndim}")
print(f"Size of U (total number of elements): {U.size}")
print(f>Data type of U elements: {U.dtype}")

Shape of U: (3, 2)
Number of dimensions of U: 2
Size of U (total number of elements): 6
Data type of U elements: float64

U_transpose = U.T
print("Transpose of U:")
display(U_transpose)

Transpose of U:

array([[2., 5., 9.],
       [7., 3., 1.]])

```

(a) Matrix of Zeros (shape (m,n))

```
zeros_matrix = np.zeros((m, n))
print(f"Matrix of zeros with shape ({m},{n}):")
display(zeros_matrix)
```

Matrix of zeros with shape (3,2):

```
array([[0., 0.],
       [0., 0.],
       [0., 0.]])
```

(b) Matrix of Ones (shape (m,n))

```
ones_matrix = np.ones((m, n))
print(f"Matrix of ones with shape ({m},{n}):")
display(ones_matrix)
```

Matrix of ones with shape (3,2):

```
array([[1., 1.],
       [1., 1.],
       [1., 1.]])
```

(c) Matrix of Random Values in [0,1] (shape (m,n))

```
random_matrix = np.random.rand(m, n)
print(f"Matrix of random values (0 to 1) with shape ({m},{n}):")
display(random_matrix)
```

Matrix of random values (0 to 1) with shape (3,2):

```
array([[0.56445016, 0.36094308],
       [0.05633388, 0.91040365],
       [0.03364536, 0.41653091]])
```

(d) Identity Matrix of order n

```
identity_matrix = np.identity(n)
print(f"Identity matrix of order {n}:")
display(identity_matrix)
```

Identity matrix of order 2:

```
array([[1., 0.],
       [0., 1.]])
```

Operations on Matrix U

(a) Maximum and Minimum Values

```
max_value_U = np.max(U)
min_value_U = np.min(U)
print(f"Maximum value in U: {max_value_U}")
print(f"Minimum value in U: {min_value_U}")
```

Maximum value in U: 9.0
Minimum value in U: 1.0

(b) Mean, Median, Variance, and Standard Deviation

```
mean_U = np.mean(U)
median_U = np.median(U)
variance_U = np.var(U)
std_dev_U = np.std(U)
```

```
print(f"Mean of U: {mean_U}")
print(f"Median of U: {median_U}")
print(f"Variance of U: {variance_U}")
print(f"Standard deviation of U: {std_dev_U}")
```

Mean of U: 4.5
Median of U: 4.0
Variance of U: 7.916666666666667
Standard deviation of U: 2.8136571693556887

(c) Row-wise Sum

```
row_sums_U = np.sum(U, axis=1)
print(f"Row-wise sums of U: {row_sums_U}")
```

Row-wise sums of U: [9. 8. 10.]

(d) Column-wise Sum

```
column_sums_U = np.sum(U, axis=0)
print(f"Column-wise sums of U: {column_sums_U}")
```

Column-wise sums of U: [16. 11.]

(e) Overall Sum

```
overall_sum_U = np.sum(U)
print(f"Overall sum of U: {overall_sum_U}")
```

Overall sum of U: 27.0

(f) Reshape U

```
U_resaped = U.reshape(n, m)
print(f"Reshaped U to shape ({n},{m}):")
display(U_resaped)
```

Reshaped U to shape (2,3):

```
array([[2., 7., 5.],
       [3., 9., 1.]])
```

(g) Flatten U into a one-dimensional array

```
U_flattened = U.flatten()
print("Flattened U:")
display(U_flattened)
```

Flattened U:

```
array([2., 7., 5., 3., 9., 1.]])
```

(h) Extract first row, last row, first column, and last column

```
first_row = U[0, :]
last_row = U[-1, :]
first_column = U[:, 0]
last_column = U[:, -1]

print("First row of U:")
display(first_row)

print("Last row of U:")
display(last_row)

print("First column of U:")
display(first_column)

print("Last column of U:")
display(last_column)
```

First row of U:

```
array([2., 7.])
```

Last row of U:

```
array([9., 1.])
```

First column of U:

```
array([2., 5., 9.])
```

Last column of U:

```
array([7., 3., 1.])
```

(i) Concatenate U with itself horizontally and vertically

```
# Horizontal concatenation
U_horizontal_concat = np.concatenate((U, U), axis=1)
print("Horizontal concatenation of U with itself:")
display(U_horizontal_concat)
```

Horizontal concatenation of U with itself:

```
array([[2., 7., 2., 7.],
       [5., 3., 5., 3.],
       [9., 1., 9., 1.]])
```

```
# Vertical concatenation
U_vertical_concat = np.concatenate((U, U), axis=0)
print("Vertical concatenation of U with itself:")
display(U_vertical_concat)
```

Vertical concatenation of U with itself:

```
array([[2., 7.],
       [5., 3.],
       [9., 1.],
       [2., 7.],
       [5., 3.],
       [9., 1.]])
```

Assume $X = U^T$

```
X = U.T
print("Matrix X (transpose of U):")
display(X)
```

Matrix X (transpose of U):

```
array([[2., 5., 9.],
       [7., 3., 1.]])
```

1. Read a positive integer p.

```
p = int(input("Enter a positive integer for p: "))

if p <= 0:
    print("p must be a positive integer.")
else:
    print(f"You entered p = {p}")
```

```
Enter a positive integer for p: 3
You entered p = 3
```

2. Create the following matrices with random values in [0,1].

(a) W of shape (p,n)

```
W = np.random.rand(p, n)
print(f"Matrix W with shape ({p},{n}):")
display(W)
```

Matrix W with shape (3,2):

```
array([[0.33029423, 0.11558071],
       [0.47395019, 0.72573645],
       [0.055628   , 0.76139425]])
```

(b) B of shape (p,1)

```
B = np.random.rand(p, 1)
print(f"Matrix B with shape ({p},1):")
display(B)
```

Matrix B with shape (3,1):

```
array([[0.76924977],
       [0.53931797],
       [0.54133111]])
```

(c) V of shape (1,p)

```
V = np.random.rand(1, p)
print(f"Matrix V with shape (1,{p}):")
display(V)
```

Matrix V with shape (1,3):

```
array([[0.86194624, 0.73566468, 0.17941671]])
```

(d) C of shape (1,1)

```
C = np.random.rand(1, 1)
print("Matrix C with shape (1,1):")
display(C)
```

Matrix C with shape (1,1):

```
array([[0.61986374]])
```

3. Compute $M1 = WX + B$

```
M1 = np.dot(W, X) + B
print("Matrix M1 (WX + B):")
display(M1)
print(f"Shape of M1: {M1.shape}")
```

Matrix M1 (WX + B):

```
array([[2.23890316, 2.76746302, 3.85747851],
       [6.56737347, 5.08627824, 5.53060608],
       [5.98234684, 3.10365385, 1.80337736]])
```

Shape of M1: (3, 3)

4. Perform element-wise multiplication between two compatible matrices (e.g., U and W)

```
element_wise_product = U * W
print("Element-wise product of U and W:")
display(element_wise_product)
```

Element-wise product of U and W:

```
array([[0.66058845, 0.80906494],
       [2.36975093, 2.17720934],
       [0.500652 , 0.76139425]])
```

5. Perform element-wise division between two compatible matrices (e.g., U and W)

```
element_wise_division = U / W
print("Element-wise division of U by W:")
display(element_wise_division)
```

Element-wise division of U by W:

```
array([[ 6.05520726, 60.56374132],
       [10.54963191,  4.13373203],
       [161.78902667,  1.31338003]])
```

6. Apply mathematical functions on M1

(a) Absolute value of M1

```
abs_M1 = np.abs(M1)
print("Absolute value of M1:")
display(abs_M1)
```

Absolute value of M1:

```
array([[2.23890316, 2.76746302, 3.85747851],
       [6.56737347, 5.08627824, 5.53060608],
       [5.98234684, 3.10365385, 1.80337736]])
```

(b) Square of M1

```
square_M1 = np.square(M1)
print("Square of M1:")
display(square_M1)
```

Square of M1:

```
array([[ 5.01268737,  7.65885156, 14.88014047],
       [43.13039433, 25.87022632, 30.58760367],
       [35.78847377,  9.63266724,  3.25216989]])
```

(c) Square root of M1

```
sqrt_M1 = np.sqrt(M1)
print("Square root of M1:")
display(sqrt_M1)
```

Square root of M1:

```
array([[1.49629648, 1.66356936, 1.96404646],
       [2.56268872, 2.25527786, 2.35172407],
       [2.44588365, 1.76171901, 1.34289886]])
```

(d) Exponential of M1

```
exp_M1 = np.exp(M1)
print("Exponential of M1:")
display(exp_M1)
```

Exponential of M1:

```
array([[ 9.38303399, 15.91819856, 47.34581883],
       [711.49861611, 161.78660896, 252.29677798],
       [396.36949508, 22.27920769,  6.07011383]])
```

(e) Natural logarithm of M1

```
log_M1 = np.log(M1)
print("Natural logarithm of M1:")
display(log_M1)
```

Natural logarithm of M1:

```
array([[0.80598609, 1.01793102, 1.35001373],
       [1.88211398, 1.62654637, 1.71029741],
       [1.78881294, 1.13258008, 0.58966122]])
```

7. Compute $M2 = VM1 + C$ using vectorized matrix multiplication

```
M2 = np.dot(V, M1) + C
print("Matrix M2 (VM1 + C):")
display(M2)
```

Matrix M2 (VM1 + C):

```
array([[8.4543956 , 7.3039107 , 8.33703044]])
```

8. Display the shapes of all intermediate matrices

```
print(f"Shape of U: {U.shape}")
print(f"Shape of U_transpose (X): {U_transpose.shape}")
print(f"Shape of zeros_matrix: {zeros_matrix.shape}")
print(f"Shape of ones_matrix: {ones_matrix.shape}")
print(f"Shape of random_matrix: {random_matrix.shape}")
print(f"Shape of identity_matrix: {identity_matrix.shape}")
print(f"Shape of U_reshaped: {U_reshaped.shape}")
print(f"Shape of U_flattened: {U_flattened.shape}")
print(f"Shape of first_row: {first_row.shape}")
print(f"Shape of last_row: {last_row.shape}")
print(f"Shape of first_column: {first_column.shape}")
print(f"Shape of last_column: {last_column.shape}")
print(f"Shape of U_horizontal_concat: {U_horizontal_concat.shape}")
print(f"Shape of U_vertical_concat: {U_vertical_concat.shape}")
print(f"Shape of W: {W.shape}")
print(f"Shape of B: {B.shape}")
print(f"Shape of V: {V.shape}")
print(f"Shape of C: {C.shape}")
print(f"Shape of M1: {M1.shape}")
print(f"Shape of element_wise_product: {element_wise_product.shape}")
print(f"Shape of element_wise_division:
{element_wise_division.shape}")
print(f"Shape of abs_M1: {abs_M1.shape}")
print(f"Shape of square_M1: {square_M1.shape}")
print(f"Shape of sqrt_M1: {sqrt_M1.shape}")
print(f"Shape of exp_M1: {exp_M1.shape}")
print(f"Shape of log_M1: {log_M1.shape}")
print(f"Shape of M2: {M2.shape}")
```

```
Shape of U: (3, 2)
Shape of U_transpose (X): (2, 3)
Shape of zeros_matrix: (3, 2)
Shape of ones_matrix: (3, 2)
Shape of random_matrix: (3, 2)
Shape of identity_matrix: (2, 2)
Shape of U_reshaped: (2, 3)
Shape of U_flattened: (6,)
Shape of first_row: (2,)
Shape of last_row: (2,)
Shape of first_column: (3,)
Shape of last_column: (3,)
Shape of U_horizontal_concat: (3, 4)
Shape of U_vertical_concat: (6, 2)
Shape of W: (3, 2)
Shape of B: (3, 1)
Shape of V: (1, 3)
Shape of C: (1, 1)
Shape of M1: (3, 3)
```

```
Shape of element_wise_product: (3, 2)
Shape of element_wise_division: (3, 2)
Shape of abs_M1: (3, 3)
Shape of square_M1: (3, 3)
Shape of sqrt_M1: (3, 3)
Shape of exp_M1: (3, 3)
Shape of log_M1: (3, 3)
Shape of M2: (1, 3)
```

9. Implement the ReLU function $f(x) = \max(0, x)$

```
def relu(X):
    """
    Applies the ReLU (Rectified Linear Unit) activation function
    element-wise to a NumPy array X.
     $f(x) = \max(0, x)$ 
    """
    return np.maximum(0, X)

print("ReLU function defined.")

ReLU function defined.
```

Example usage of the `relu` function:

```
test_matrix = np.array([[ -1,  2, -3], [ 4, -5,  6], [ 0, -7,  8]])
print("Original test matrix:")
display(test_matrix)

relu_output = relu(test_matrix)
print("ReLU output:")
display(relu_output)

Original test matrix:
array([[ -1,  2, -3],
       [  4, -5,  6],
       [  0, -7,  8]])

ReLU output:
array([[0, 2, 0],
       [4, 0, 6],
       [0, 0, 8]])
```

10. Implement the Sigmoid function $g(x) = 1 / (1 + e^{-x})$

```
def sigmoid(X):
    """
    Applies the Sigmoid activation function element-wise to a NumPy
    array X.
    """
```

```

    g(x) = 1 / (1 + e^-x)
    """
    return 1 / (1 + np.exp(-X))

print("Sigmoid function defined.")
Sigmoid function defined.

```

Example usage of the `sigmoid` function:

```

test_matrix_sigmoid = np.array([[ -1, 0, 1], [-2, 0.5, 2]])
print("Original test matrix for sigmoid:")
display(test_matrix_sigmoid)

sigmoid_output = sigmoid(test_matrix_sigmoid)
print("Sigmoid output:")
display(sigmoid_output)

Original test matrix for sigmoid:
array([[ -1. ,  0. ,  1. ],
       [-2. ,  0.5,  2. ]])

Sigmoid output:
array([[0.26894142, 0.5      , 0.73105858],
       [0.11920292, 0.62245933, 0.88079708]])

```

11. Compute $Z1 = WX + B$ and apply the ReLU function $A1 = \text{relu}(Z1)$

```

Z1 = M1 # Z1 is equivalent to M1 = WX + B
print("Matrix Z1 (WX + B):")
display(Z1)

A1 = relu(Z1)
print("Matrix A1 (relu(Z1)):")
display(A1)

Matrix Z1 (WX + B):
array([[2.23890316, 2.76746302, 3.85747851],
       [6.56737347, 5.08627824, 5.53060608],
       [5.98234684, 3.10365385, 1.80337736]])

Matrix A1 (relu(Z1)):
array([[2.23890316, 2.76746302, 3.85747851],
       [6.56737347, 5.08627824, 5.53060608],
       [5.98234684, 3.10365385, 1.80337736]])

```

12. Compute $Z2 = VA1 + C$ and apply the Sigmoid function $A2 = \text{sigmoid}(Z2)$

```
Z2 = np.dot(V, A1) + C
print("Matrix Z2 (VA1 + C):")
display(Z2)

A2 = sigmoid(Z2)
print("Matrix A2 (sigmoid(Z2)):")
display(A2)

Matrix Z2 (VA1 + C):
array([[8.4543956 , 7.3039107 , 8.33703044]])

Matrix A2 (sigmoid(Z2)):
array([[0.99978708, 0.99932755, 0.99976057]])
```

13. Display $Z1$, $A1$, $Z2$, and $A2$

(a) $Z1$

```
print("Z1:")
display(Z1)

Z1:
array([[2.23890316, 2.76746302, 3.85747851],
       [6.56737347, 5.08627824, 5.53060608],
       [5.98234684, 3.10365385, 1.80337736]])
```

(b) $A1$

```
print("A1:")
display(A1)

A1:
array([[2.23890316, 2.76746302, 3.85747851],
       [6.56737347, 5.08627824, 5.53060608],
       [5.98234684, 3.10365385, 1.80337736]])
```

(c) $Z2$

```
print("Z2:")
display(Z2)

Z2:
array([[8.4543956 , 7.3039107 , 8.33703044]])
```

(d) A2

```
print("A2:")  
display(A2)
```

A2:

```
array([[0.99978708, 0.99932755, 0.99976057]])
```

14. Repeat the computations for 10 iterations with newly generated random matrices

```
for i in range(1, 11):  
    print(f"\n--- Iteration {i} ---")  
  
    # Generate new random matrices for each iteration  
    W = np.random.rand(p, n)  
    B = np.random.rand(p, 1)  
    V = np.random.rand(1, p)  
    C = np.random.rand(1, 1)  
  
    # Recompute M1 = WX + B  
    M1 = np.dot(W, X) + B  
  
    # Recompute Z1 = WX + B  
    Z1 = M1  
  
    # Recompute A1 = relu(Z1)  
    A1 = relu(Z1)  
  
    # Recompute Z2 = VA1 + C  
    Z2 = np.dot(V, A1) + C  
  
    # Recompute A2 = sigmoid(Z2)  
    A2 = sigmoid(Z2)  
  
    # Display the results for the current iteration  
    print("Z1:")  
    display(Z1)  
    print("A1:")  
    display(A1)  
    print("Z2:")  
    display(Z2)  
    print("A2:")  
    display(A2)
```

--- Iteration 1 ---

Z1:

```
array([[7.11880567, 7.21476098, 9.47539057],  
       [2.24151728, 3.19345917, 4.84601173],  
       [6.46670783, 4.60591513, 4.52416779]])
```

A1:

```
array([[7.11880567, 7.21476098, 9.47539057],  
       [2.24151728, 3.19345917, 4.84601173],  
       [6.46670783, 4.60591513, 4.52416779]])
```

Z2:

```
array([[7.10741109, 5.88912102, 6.4983173 ]])
```

A2:

```
array([[0.99918166, 0.99723824, 0.99849629]])
```

--- Iteration 2 ---

Z1:

```
array([[4.16010254, 2.79141386, 2.68192518],  
       [4.77660901, 5.95294402, 8.74106857],  
       [3.45662709, 4.99881227, 7.81613874]])
```

A1:

```
array([[4.16010254, 2.79141386, 2.68192518],  
       [4.77660901, 5.95294402, 8.74106857],  
       [3.45662709, 4.99881227, 7.81613874]])
```

Z2:

```
array([[3.52372872, 3.93889295, 5.37772   ]])
```

A2:

```
array([[0.97135543, 0.98090208, 0.99540289]])
```

--- Iteration 3 ---

Z1:

```
array([[8.12420781, 7.08697336, 8.73555685],  
       [1.11232917, 0.88302951, 0.8876155 ],  
       [8.48551063, 6.41017261, 6.91906134]])
```

A1:

```
array([[8.12420781, 7.08697336, 8.73555685],  
       [1.11232917, 0.88302951, 0.8876155 ],  
       [8.48551063, 6.41017261, 6.91906134]])
```

Z2:

```
array([[7.31322692, 5.88130883, 6.32971677]])
```

A2:

```
array([[0.99933378, 0.99721664, 0.99822063]])
```

--- Iteration 4 ---

Z1:

```
array([[7.02624182, 4.99753637, 5.13589218],
       [4.75536111, 4.97042341, 6.62447186],
       [3.31772352, 4.45235772, 6.51847019]])
```

A1:

```
array([[7.02624182, 4.99753637, 5.13589218],
       [4.75536111, 4.97042341, 6.62447186],
       [3.31772352, 4.45235772, 6.51847019]])
```

Z2:

```
array([[4.07165645, 4.50075285, 5.88006754]])
```

A2:

```
array([[0.98323668, 0.98902124, 0.99721319]])
```

--- Iteration 5 ---

Z1:

```
array([[6.4483154 , 5.29979227, 5.9210677 ],
       [6.71273882, 3.32223386, 1.62886023],
       [5.63126925, 4.97111271, 5.8889163 ]])
```

A1:

```
array([[6.4483154 , 5.29979227, 5.9210677 ],
       [6.71273882, 3.32223386, 1.62886023],
       [5.63126925, 4.97111271, 5.8889163 ]])
```

Z2:

```
array([[12.1951343 ,  8.76749068,  8.40249219]])
```

A2:

```
array([[0.99999495, 0.99984431, 0.99977574]])
```

--- Iteration 6 ---

Z1:

```
array([[3.33904594, 4.01219638, 5.84928431],
       [6.46833196, 3.22414919, 1.67243157],
       [2.83919348, 4.6134977 , 7.46283624]])
```

A1:

```
array([[3.33904594, 4.01219638, 5.84928431],
       [6.46833196, 3.22414919, 1.67243157],
       [2.83919348, 4.6134977 , 7.46283624]])
```

Z2:

```
array([[3.99249806, 4.4100381 , 5.83538375]])
```

A2:

```
array([[0.98188081, 0.98799125, 0.99708621]])
```

--- Iteration 7 ---

Z1:

```
array([[5.77417444, 4.87286562, 5.64919284],
       [2.73428082, 1.6445954 , 1.35208911],
       [5.37878177, 5.74000984, 7.65437529]])
```

A1:

```
array([[5.77417444, 4.87286562, 5.64919284],
       [2.73428082, 1.6445954 , 1.35208911],
       [5.37878177, 5.74000984, 7.65437529]])
```

Z2:

```
array([[9.80461687, 8.11718809, 9.14335023]])
```

A2:

```
array([[0.99994481, 0.99970172, 0.99989308]])
```

--- Iteration 8 ---

Z1:

```
array([[6.93834626, 3.71278003, 2.25655588],
       [3.2934285 , 4.30726467, 6.49500419],
       [4.90613386, 3.63570124, 3.91320149]])
```

A1:

```
array([[6.93834626, 3.71278003, 2.25655588],
       [3.2934285 , 4.30726467, 6.49500419],
       [4.90613386, 3.63570124, 3.91320149]])
```

Z2:

```
array([[7.72215835, 6.35201926, 7.2465853 ]])
```

A2:

```
array([[0.99955729, 0.99825981, 0.9992879 ]])
```

--- Iteration 9 ---

Z1:

```
array([[5.74425424, 4.70541497, 5.44623771],  
       [8.44732974, 7.21406217, 8.46913114],  
       [6.81421317, 4.93734984, 5.03236824]])
```

A1:

```
array([[5.74425424, 4.70541497, 5.44623771],  
       [8.44732974, 7.21406217, 8.46913114],  
       [6.81421317, 4.93734984, 5.03236824]])
```

Z2:

```
array([[15.15671473, 12.07866178, 13.31142041]])
```

A2:

```
array([[0.99999974, 0.99999432, 0.99999834]])
```

--- Iteration 10 ---

Z1:

```
array([[2.80941169, 1.98799151, 1.87872122],  
       [5.75004639, 4.84728704, 5.5201595 ],  
       [3.87653945, 4.46150855, 6.36357076]])
```

A1:

```
array([[2.80941169, 1.98799151, 1.87872122],  
       [5.75004639, 4.84728704, 5.5201595 ],  
       [3.87653945, 4.46150855, 6.36357076]])
```

Z2:

```
array([[7.89867737, 7.28894506, 8.7946054 ]])
```

A2:

```
array([[0.9996289 , 0.99931742, 0.99984847]])
```